Author: Mohammad Toki Bin Alam

Technical Design Report

1. Design Approach & Executive Summary

I tried to implement a digital circuit breaker using an **ESP32 DevKit C V4** to protect electrical appliances from grid voltage surges, undervoltage brownouts, and overcurrent faults.

Dynamic configuration is provided via a dual-interface architecture (Serial CLI and Wi-Fi REST API).

2. Circuit Mapping & Hardware Architecture

- **Main Controller:** ESP32 DevKit C V4 (3.3V logic, 12-bit ADC).
- **Sensor Emulation:** Slide Potentiometer 1 (GPIO 34) scales $0 \text{ -- } 3.3V$ to represent $0 \text{ -- } 300V\ AC$ grid voltage. Slide Potentiometer 2 (GPIO 35) scales $0 \text{ -- } 3.3V$ to represent $0 \text{ -- } 30A\ AC$ load current.
- **Actuation & Load:** GPIO 4 controls a 5V SPDT Relay module. An Orange LED (led1) powered through the Relay's Normally Open (NO) terminal serves as the physical load to visually confirm circuit tripping.
- **User Interface:** I2C 16x2 LCD (GPIO 21 SDA, GPIO 22 SCL) displays real-time values and status (SAFE / TRIPPED). A Green LED (GPIO 18) and Red LED (GPIO 19) indicate operational status.

3. Control Logic & Communication Modes

The ESP32 evaluates live readings against memory-stored safety limits (Defaults: $V_{MAX} = 250V$, $V_{MIN} = 180V$, $I_{MAX} = 15.0A$):

$$SystemFault = (V_{grid} > V_{MAX}) \vee (V_{grid} < V_{MIN}) \vee (I_{load} > I_{MAX})$$

When a fault occurs, GPIO 4 pulls LOW, physically opening the relay contacts and disconnecting the Orange LED load. Settings can be updated live via:

1. **Serial CLI (Active Simulation):** Accepts UART text commands (SET_VMAX <val>, SET_VMIN <val>, SET_IMAX <val>, RESET).
2. **REST API (Hardware Deployment):** Binds HTTP GET endpoints via <WiFi.h> and <WebServer.h> (/api/set?vmax=270&vmin=170&imax=20).

4. Limitations & Engineering Solutions

- **DC Potentiometer vs. Real AC Mains:** Potentiometers output DC, whereas grid power is AC. *Solution:* In a physical build, use a **ZMPT101B** potential transformer and an **ACS712 / CT Clamp** with true RMS software sampling over $20ms$ AC cycles.
- **Browser Sandbox Network Constraints:** Wokwi limits local network bridging for browser-hosted REST APIs without a paid gateway. *Solution:* Implemented a dual-mode firmware containing both Serial CLI (for simulation testing) and WebServer handlers (for hardware deployment).